DATA STRUCTURES & ALGORITHMS

Simple, Human-Friendly Answers – Sets 1, 2 and 3

All programs use Python 3. Each example is self-contained and can be run separately.

SET 1 – E-Commerce Catalogue and Traffic Analysis

Scenario in simple words: An online shopping website needs to organise product categories, find products quickly, understand which services depend on other services, and prepare for busy traffic.

1. Binary Search Tree (BST) for 15 product categories

Simple idea: A BST stores values in an ordered tree. Smaller values go to the left and larger values go to the right. Inorder traversal prints the values in sorted order; preorder and postorder show different visiting orders.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root

def search(root, value):
    if root is None or root.value == value:
        return root is not None
    if value < root.value:
        return search(root.left, value)
    return search(root.right, value)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.value, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.value, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.value, end=" ")

categories = ["Books", "Clothing", "Electronics", "Furniture",
```

```
             "Groceries", "Beauty", "Toys", "Sports",
             "Shoes", "Jewellery", "Watches", "Kitchen",
             "Mobile", "Laptop", "Stationery"]

root = None
for category in categories:
    root = insert(root, category)

print("Inorder:"); inorder(root)
print("\nPreorder:"); preorder(root)
print("\nPostorder:"); postorder(root)
print("\nSearch Mobile:", search(root, "Mobile"))
```

Example output: All three traversals print the same 15 categories in different orders. Inorder prints them alphabetically. Search Mobile: True.

Time complexity: Average insert/search: O(log n); worst case: O(n). Each traversal: O(n).

2. Hash table with chaining and built-in dictionary

Simple idea: A hash table converts a SKU into an array position. A collision happens when two SKUs get the same position. Chaining keeps both values in a list at that position. Python's dictionary already handles collisions internally.

```
# Small hash table using chaining
size = 5
table = [[] for _ in range(size)]

def put(key, value):
    index = sum(ord(ch) for ch in key) % size
    for pair in table[index]:
        if pair[0] == key:
            pair[1] = value
            return
    table[index].append([key, value])

def get(key):
    index = sum(ord(ch) for ch in key) % size
    for k, value in table[index]:
        if k == key:
            return value
    return None

put("SKU10", "Phone")
put("SKU01", "Headphones")
put("SKU20", "Laptop")
print("Hash table:", table)
print("SKU10:", get("SKU10"))

# Same lookup task using Python's built-in hash map (dict)
sku_map = {f"SKU{i}": f"Product{i}" for i in range(100000)}
print("Dictionary lookup:", sku_map.get("SKU99999"))
```

Example output: At least one bucket may contain more than one SKU; that is the collision. The dictionary returns Product99999 for SKU99999.

Time complexity: Average insert/search: O(1); worst case: O(n). The dictionary is generally the practical choice. A 100,000-item timing test depends on the computer, so use timeit for a fair measured comparison.

3. Microservice dependencies using a graph, BFS/DFS and cycle detection

Simple idea: A directed graph shows which service must be ready before another service. If A points to B, A is a dependency of B. Topological sorting gives a valid order. A cycle means the services depend on each other in a loop.

```
from collections import deque

graph = {
    "cart": ["pricing"],
    "pricing": ["inventory"],
    "inventory": ["payment"],
    "payment": ["notification"],
    "notification": []
}

# BFS traversal
def bfs(graph, start):
    q = deque([start])
    seen = {start}
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                q.append(nxt)
    return order

# DFS traversal
def dfs(graph, start, seen=None):
    if seen is None:
        seen = set()
    seen.add(start)
    print(start, end=" ")
    for nxt in graph[start]:
        if nxt not in seen:
            dfs(graph, nxt, seen)

# Topological order + cycle detection
def topological_sort(graph):
    state = {node: 0 for node in graph}  # 0=new, 1=visiting, 2=done
    order = []
    def visit(node):
        if state[node] == 1:
            return False
        if state[node] == 2:
            return True
        state[node] = 1
        for nxt in graph[node]:
            if not visit(nxt):
                return False
        state[node] = 2
        order.append(node)
        return True
    for node in graph:
        if state[node] == 0 and not visit(node):
            return None
    return order[::-1]

print("BFS:", bfs(graph, "cart"))
print("DFS:", end=" "); dfs(graph, "cart")
print("\nDeployment order:", topological_sort(graph))

# Add a circular dependency to test detection
graph["notification"] = ["cart"]
print("Order with cycle:", topological_sort(graph))
```

Example output: The first graph gives cart → pricing → inventory → payment → notification. After adding notification → cart, the result is None, meaning a cycle was found.

Time complexity: BFS/DFS/topological sorting: O(V + E), where V is services and E is dependency links.

4. Busiest continuous 3-hour window (sliding window)

Simple idea: Since each value represents one hour, a 3-hour window contains three neighbouring values. Calculate the first sum, then slide one hour at a time by removing the old value and adding the new one.

```
requests = [10, 15, 12, 30, 25, 20, 18, 40, 35, 22,
            17, 19, 21, 23, 28, 31, 26, 24, 20, 18,
            16, 14, 12, 11]

k = 3
window_sum = sum(requests[:k])
best_sum = window_sum
best_start = 0

for i in range(k, len(requests)):
    window_sum += requests[i] - requests[i-k]
    if window_sum > best_sum:
        best_sum = window_sum
        best_start = i-k+1

print("Busiest 3-hour window:",
      requests[best_start:best_start+k])
print("Total requests:", best_sum)
print("Starting hour (0-based):", best_start)
```

Example output: The program prints the three consecutive hourly counts with the largest total. The starting hour is shown using zero-based indexing.

Time complexity: O(n), because each hour is added and removed at most once.

5. Two pointers to find order pairs equal to a gift-card value

Simple idea: The list must be sorted. Keep one pointer at the beginning and one at the end. If the sum is too small, move the left pointer right. If it is too large, move the right pointer left.

```
orders = [10, 20, 30, 40, 50, 60, 70]
gift_card = 80

left, right = 0, len(orders) - 1
while left < right:
    total = orders[left] + orders[right]
    if total == gift_card:
        print(orders[left], "+", orders[right], "=", gift_card)
        left += 1
        right -= 1
    elif total < gift_card:
        left += 1
    else:
        right -= 1
```

Example output: Matching pairs: 10 + 70, 20 + 60, and 30 + 50.

Time complexity: O(n) time and O(1) extra space, excluding output.

SET 2 – Insurance Claims Processing

Scenario in simple words: An insurance company needs to organise claim IDs, identify repeated claims, understand approval-stage connections, and monitor unusually high claim counts.

1. BST for claim IDs and traversals

Simple idea: Insert each claim ID into a BST. Inorder traversal visits the left side, the node, and then the right side. This produces claim IDs in ascending order.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.value, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.value, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.value, end=" ")

claims = [105, 102, 110, 101, 104, 108, 115]
root = None
for claim in claims:
    root = insert(root, claim)

print("Inorder:"); inorder(root)
print("\nPreorder:"); preorder(root)
print("\nPostorder:"); postorder(root)
```

Example output: Inorder: 101 102 104 105 108 110 115. This is the traversal that gives sorted claim IDs.

Time complexity: Average insertion: O(log n); worst case: O(n). Traversal: O(n).

2. Hashing to detect duplicate or near-duplicate claims

Simple idea: Make a standard key from the policy number and incident date. Removing spaces and ignoring capital letters helps catch small formatting differences. A set can then detect whether the key has appeared before. A real system should also use careful rules so genuinely different claims are not incorrectly treated as duplicates.

```
claims = [
    ("POL123", "2026-10-01"),
    (" pol123 ", "2026-10-01"),
    ("POL456", "2026-10-02"),
    ("POL123", "2026-10-01 ")
]

seen = set()
for policy, date in claims:
    key = (policy.strip().upper(), date.strip())
    if key in seen:
        print("Duplicate claim:", key)
    else:
        seen.add(key)
        print("New claim:", key)
```

Example output: The second and fourth entries are detected as duplicates because their cleaned policy number and date match the first entry.

Time complexity: Average lookup/insert: O(1); worst case: O(n).

3. Claim approval stages as a directed graph

Simple idea: Each arrow shows a possible next approval stage. BFS finds a path with the fewest edges when every step has equal cost. DFS can be used to detect a loop in the approval process.

```
from collections import deque

graph = {
    "Registration": ["Assessment"],
    "Assessment": ["Surveyor", "Settlement"],
    "Surveyor": ["Settlement"],
    "Settlement": []
}

def bfs_path(graph, start, goal):
    q = deque([(start, [start])])
    seen = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, path + [nxt]))
    return None

def has_cycle(graph):
    visiting, done = set(), set()
    def visit(node):
        if node in visiting:
            return True
        if node in done:
            return False
        visiting.add(node)
        for nxt in graph[node]:
            if visit(nxt):
                return True
        visiting.remove(node)
        done.add(node)
        return False
    return any(visit(n) for n in graph if n not in done)
```

```
print("Shortest path:",
      bfs_path(graph, "Registration", "Settlement"))
print("Has loop:", has_cycle(graph))

# Add a loop to test
graph["Settlement"] = ["Assessment"]
print("Has loop after change:", has_cycle(graph))
```

Example output: Shortest path: Registration → Assessment → Settlement. Initially Has loop: False. After adding Settlement → Assessment, Has loop: True.

Time complexity: BFS and DFS cycle detection: O(V + E).

4. Longest consecutive run of days with more than 100 claims

Simple idea: First convert each day into True (more than 100 claims) or False. Then scan the list and count consecutive True values. Reset the count when a day does not cross 100.

```
daily_claims = [90, 120, 130, 115, 80, 140, 150, 160, 70, 110]

current = best = 0
for count in daily_claims:
    if count > 100:
        current += 1
        best = max(best, current)
    else:
        current = 0

print("Longest run:", best, "days")
```

Example output: Output: Longest run: 3 days.

Time complexity: O(n) time and O(1) extra space.

5. Merge two sorted regional claim lists using two pointers

Simple idea: Start one pointer in each sorted list. Compare the two current values and copy the smaller one into the result. When one list finishes, add the remaining values from the other list.

```
north = [101, 104, 108, 115]
south = [102, 103, 110, 120]

i = j = 0
merged = []

while i < len(north) and j < len(south):
    if north[i] <= south[j]:
        merged.append(north[i])
        i += 1
    else:
        merged.append(south[j])
        j += 1

merged.extend(north[i:])
merged.extend(south[j:])

print("Merged claims:", merged)
```

Example output: Output: [101, 102, 103, 104, 108, 110, 115, 120].

Time complexity: O(n + m) time, where n and m are the lengths of the two lists.

SET 3 – Logistics Network and Delivery Windows

Scenario in simple words: A courier company needs to organise hub codes, track parcels, find routes between hubs, identify busy delivery hours, and validate route descriptions.

1. BST for hub codes and tree height

Simple idea: Store hub codes in a BST. The height is the number of edges on the longest path from the root to a leaf. Sorted input can create a very one-sided tree, making it behave like a linked list.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root

def height(root):
    if root is None:
        return -1
    return 1 + max(height(root.left), height(root.right))

# Example insertion order
codes = [50, 30, 70, 20, 40, 60, 80]
root = None
for code in codes:
    root = insert(root, code)

print("Search/insert example: hub 60 is in the tree")
print("Tree height:", height(root))

# Sorted input creates a skewed tree
sorted_root = None
for code in [10, 20, 30, 40, 50]:
    sorted_root = insert(sorted_root, code)
print("Height for sorted input:", height(sorted_root))
```

Example output: Balanced-looking example height: 2. Sorted-input example height: 4. Sorted input makes this BST skewed.

Time complexity: Average insert/search: O(log n); worst case: O(n). Height calculation: O(n).

2. Hash map for parcel tracking and repeated hub scans

Simple idea: Use a dictionary to store the latest hub scanned for each parcel. If the parcel is scanned again at the same hub, report the repeated scan. In a real tracking system, timestamps should also be checked.

```
scans = [
    ("PKG101", "Chennai"),
    ("PKG102", "Bengaluru"),
    ("PKG101", "Chennai"),
    ("PKG101", "Hyderabad"),
    ("PKG102", "Bengaluru")
]

last_hub = {}
for parcel, hub in scans:
    if parcel in last_hub and last_hub[parcel] == hub:
        print(parcel, "scanned twice at", hub)
    else:
        print(parcel, "scan recorded at", hub)
    last_hub[parcel] = hub
```

Example output: PKG101 is reported twice at Chennai, and PKG102 is reported twice at Bengaluru.

Time complexity: Average dictionary operation: O(1); worst case: O(n).

3. Ten hubs as an adjacency list; BFS and DFS

Simple idea: An adjacency list stores each hub and the hubs directly connected to it. BFS finds the route with the fewest hops in an unweighted graph. DFS explores as far as possible along one route before backtracking.

```
from collections import deque

graph = {
    "H1": ["H2", "H3"],
    "H2": ["H1", "H4", "H5"],
    "H3": ["H1", "H6"],
    "H4": ["H2", "H7"],
    "H5": ["H2", "H8"],
    "H6": ["H3", "H9"],
    "H7": ["H4", "H10"],
    "H8": ["H5"],
    "H9": ["H6"],
    "H10": ["H7"]
}

def shortest_route(graph, start, goal):
    q = deque([(start, [start])])
    seen = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, path + [nxt]))
    return None

def dfs(graph, node, seen):
    seen.add(node)
    print(node, end=" ")
    for nxt in graph[node]:
        if nxt not in seen:
            dfs(graph, nxt, seen)

print("Fewest-hop route H1 to H10:",
      shortest_route(graph, "H1", "H10"))
print("Hubs reachable from H1:", end=" ")
dfs(graph, "H1", set())
```

Example output: Fewest-hop route: H1 → H2 → H4 → H7 → H10. DFS prints all ten hubs because all are connected.

Time complexity: BFS/DFS: O(V + E).

4. Four-hour window with the highest average parcel count

Simple idea: For a fixed four-hour window, the average is the total divided by four. Since every window has the same size, the window with the highest average is also the one with the highest total. Sliding the window avoids recalculating each sum from scratch.

```
parcels = [12, 18, 20, 15, 30, 28, 25, 19, 16, 22]
k = 4

total = sum(parcels[:k])
best_total = total
best_start = 0

for i in range(k, len(parcels)):
    total += parcels[i] - parcels[i-k]
    if total > best_total:
        best_total = total
        best_start = i-k+1

window = parcels[best_start:best_start+k]
print("Best 4-hour window:", window)
print("Highest average:", best_total / k)
```

Example output: The program displays the four consecutive hours with the highest average parcel count.

Time complexity: O(n) time and O(1) extra space.

5. Check whether a route description is a palindrome

Simple idea: A palindrome reads the same forward and backward. Use two pointers, one at each end. Ignore spaces, hyphens and letter case. Move both pointers towards the centre while comparing characters.

```
def is_palindrome(text):
    cleaned = "".join(ch.lower() for ch in text if ch.isalnum())
    left, right = 0, len(cleaned) - 1

    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True

route = "H1 - H2 - H1"
print("Is palindrome:", is_palindrome(route))

route2 = "H1 - H2 - H3"
print("Is palindrome:", is_palindrome(route2))
```

Example output: First route: True. Second route: False. The check ignores separators and capital letters.

Time complexity: O(n) time. The cleaned string uses O(n) extra space.

Quick Decision Tables

These tables summarise which data structure or pattern is suitable and why.

Set 1 – E-Commerce

Data structure / pattern	Used for	Why it fits
BST	Store categories in sorted tree form	Supports insert, search and ordered traversal
Hash table / dict	Find a SKU quickly	Average O(1) lookup; chaining handles collisions
Graph + BFS/DFS	Represent service dependencies	Shows connections, traversal and cycles
Sliding window	Find busiest 3-hour period	Updates a fixed-size total efficiently
Two pointers	Find two order values matching a target	Uses sorted order; avoids checking every pair

Set 2 – Insurance

Data structure / pattern	Used for	Why it fits
BST	Store claim IDs	Inorder traversal gives sorted IDs
Hash set	Detect duplicate claims	Fast average membership checking
Directed graph + BFS/DFS	Represent approval stages	BFS finds fewest steps; DFS detects loops
Sliding window / scan	Find consecutive high-claim days	Counts a continuous run in one pass
Two pointers	Merge sorted regional lists	Merges without sorting the combined list again

Set 3 – Logistics

Data structure / pattern	Used for	Why it fits
BST	Store hub codes	Supports search/insert; height shows tree shape
Hash map	Track parcel scans	Fast parcel-to-hub lookup
Adjacency list + BFS/DFS	Represent hub routes	BFS finds fewest hops; DFS visits reachable hubs
Sliding window	Find highest 4-hour average	Efficient fixed-size window comparison
Two pointers	Check route palindrome	Compares matching characters from both ends

Complexity Cheat Sheet

Operation	Time complexity
BST search/insert	O(log n) average; O(n) worst case
BST traversal / height	O(n)
Hash table / dictionary	O(1) average; O(n) worst case
BFS / DFS / topological sort	O(V + E)
Sliding window	O(n)
Two pointers	Usually O(n) for sorted input or palindrome checks
Merge two sorted lists	O(n + m)

Important points to remember

- BST inorder = left, root, right. It prints values in ascending order when the BST uses normal numeric or alphabetical comparison.
- A hash collision means different keys land in the same bucket. Chaining stores multiple entries in that bucket.
- BFS uses a queue and is useful for the fewest number of edges in an unweighted graph.
- DFS goes deep before backtracking. It is useful for exploring connected nodes and detecting cycles.
- A directed graph has arrows. A cycle means following arrows can bring you back to a node already in the current path.
- Sliding window is best when checking consecutive groups of a fixed size.
- Two pointers are especially useful with sorted lists and when comparing the two ends of a sequence.